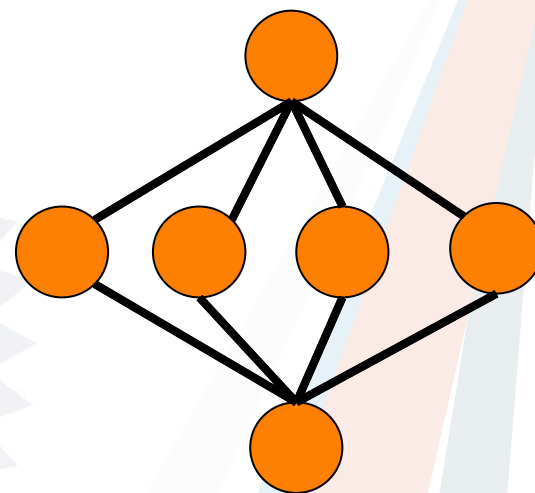
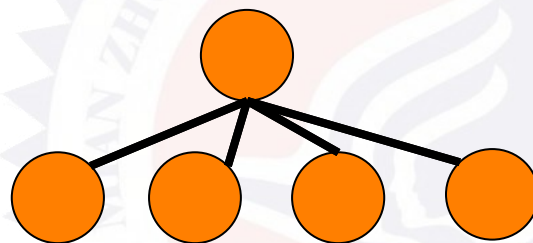


树形数据结构 ——线段树

主讲：黄凤林

数据结构

维护数据与数据之间的关系的存储结构



线性类数据结构：

数组：任意位置访问存储

栈：一端操作（查询、插入、删除等）

队列：两端操作（队尾进、队首出）

链表：每次查询从头开始，插入和删除比较方便

树形数据结构

- **数据结构除了线性以外，其他的节本都是树形的。**
- 树形的数据结构众多，基本上都是用完全二叉树的形态来构建的，也就是如果**节点编号为 i** ，则其**左孩子编号为 $2*i$** ，**右孩子为 $2*i+1$** ，其父节点编号为 $i/2$ ；
- 因此，我们就可以利用数组来模拟构建树形数据结构
- 常见的树形数据结构有：
- 堆（优先队列）、并查集、树状数组、线段树、字典树



区间问题

一. 只查询不修改

- ① n 个数, m 次查询, 求前缀和
- ② n 个数, m 次查询, 求区间 $[a,b]$ 之和
- ③ n 个数, m 次查询, 求前 i 个数的极值
- ④ n 个数, m 次查询, 求区间 $[a,b]$ 的极值
- ⑤ n 个数, m 次查询, 求固定长度的所有区间的极值

➔ 区间极值问题--RMQ

- **RMQ**是英文Range Maximum(Minimum)Query的缩写
- 问题是指：对于长度为 n 的数列 A ，回答若干询问 $RMQ(A, i, j)$ ($i, j \leq n$)，返回数列 A 中下标在 i, j 里的最小（大）值，也就是说，**RMQ问题是指求区间最值的问题。**
- 今天主要讲解的是RMQ的ST算法(倍增)，ST算法它的本质相当于是**动态规划，只是对普通动态规划的优化**
- ST算法的流程：**1、预处理；2、询问；3、技巧**



→ ST算法

- 前言
- 思考这样子一个问题：给定一个长度为 N 的序列，有 M 个询问，每次询问给你两个数字 X, Y 。请你回答出 X 到 Y 这段区间累的最大数。
- 数据范围
 - 30%的数据， $n, m \leq 1000$
 - 100%的数据， $n, m \leq 100000$

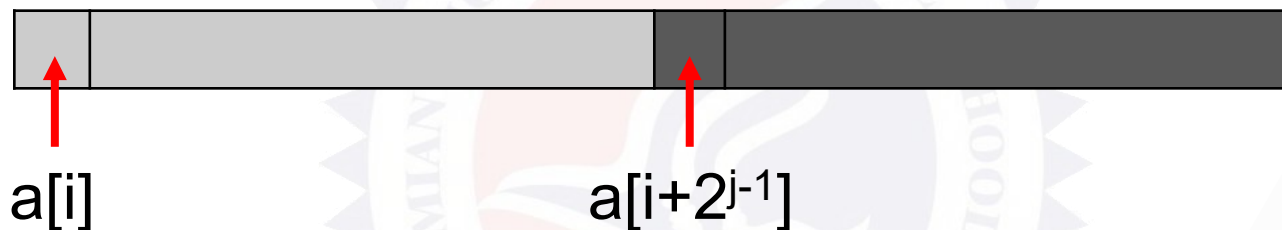


→ ST算法

- 问题分析
- 针对30%的数据，由于 n, m 很小且这里只存在查询，不存在修改，所以我们很容易想到一个暴力方法：把任意两个位置区间最值预处理出来， **$dp[i][j]$ 表示 $i \sim j$ 位置的最大值**，则查询起来只需要 $O(1)$ 输出即可。
- 则很容易得到： **$dp[i][j] = \max(dp[i][j-1], a[j])$** 。
- 这样子预处理的时间复杂度和空间复杂度均为 N^2 。
- 对于100%的数据是不得行。因此我们就必须要寻求更好的方法，也就是我们要讲的RMQ ST算法

ST算法

- **第一步：预处理**
- 用 $dp[i][j]$ 表示从 $a[i]$ 到 $a[i+2^j-1]$ 这个范围的最大值，则中间有 2^j 个元素，也就是以 $a[i]$ 为起点连续 2^j 个数的最大值。由于元素个数为 2^j 个，所以从中间平均分成两部分，每一部分的元素刚好为 2^{j-1} 个，也就是说，我们把 $dp[i][j]$ 分成 $dp[i, j-1]$ 和 $dp[i+2^{j-1}, j-1]$ 两部分。



- 整个区间的最大值一定是最优两部分最大值的较大值，满足动态规划的最优化原理，所以 $dp[i][j] = \max(dp[i][j-1], dp[i+2^{j-1}][j-1])$ ，边界条件为 $dp[i][0] = a[i]$ ，这样我们就可以在 $O(n \log n)$ 的时间复杂度内预处理 dp 数组

➔ 预处理代码实现

```
const int maxn=100000+10;
const int len=20;
int n,m,a[maxn];
int dp[maxn][len]; //dp[i][j]表示从i开始连续2^j个元素的最大值
int max(int x,int y){
    return x>=y?x:y;
}
int main()
{
    int x,y;
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++){ scanf("%d",&a[i]); dp[i][0]=a[i]; }
    for(int j=1;j<=len;j++){
        for(int i=1;(i+(1<<j)-1)<=n;i++){
            dp[i][j]=max(dp[i][j-1],dp[i+(1<<(j-1))][j-1]);
        }
    }
}
```



ST算法

➤ 第二步：查询

- 在我们要查询区间 $[x,y]$ 的最大值。则先求出最大的 k 满足 $2^k \leq y-x+1$ ，那么区间 $[x,y] = [x, x+2^k-1] \cup [y-2^k+1, y]$ 。

区间 $= [x, x+2^k-1]$



区间 $= [y-2^k+1, y]$

- 由于两个区间的元素均为 2^k 个，因此 $[x, x+2^k-1]$ 和 $[y-2^k+1, y]$ 两个区间一定可以覆盖整个 $[x,y]$ 区间，虽然这两个区间有交集，但对于求最值来说没有影响，也就是ST算法只能适用于求区间最值的原因。

➔ 查询参考代码

- 查询参考代码很简单，求区间 $[x,y]$,直接给出表达式：

$K = \log(y-x+1)/\log(2);$

$ans = \max(dp[x][k], dp[y-2^k+1][k]);$

- 技巧：由于cmath库中log效率不高，所以我们通常还可以用 $O(N)$ 的递推预处理出区间长度为 $1 \sim N$ 各自所对应的k值。具体的,设 $lg[d]$ 表示 $\log_2 d$ 向下取整，则：

$lg[d] = lg[d/2] + 1;$

```
for(int i=1;i<=m;i++){
    scanf("%d%d",&x,&y);
    int k=log(y-x+1)/log(2);
    printf("%d\n",max(dp[x][k],dp[y-(1<<k)+1][k]));
}
```

→ P1816 忠诚

➤ 题目描述

老管家是一个聪明能干的人。他为财主工作了整整10年，财主为了让自已账目更加清楚。要求管家每天记 k 次账，由于管家聪明能干，因而管家总是让财主十分满意。但是由于一些人的挑拨，财主还是对管家产生了怀疑。于是他决定用一种特别的方法来判断管家的忠诚，他把每次的账目按1, 2, 3...编号，然后不定时的问管家问题，问题是这样的：在 a 到 b 号账中最少的一笔是多少？为了让管家没时间作假他总是一次问多个问题。

➔ 最敏捷的机器人

➤ 问题描述

Wind 设计了很多机器人。但是它们都认为自己是最强的，于是，一场比赛开始了……

机器人们都想知道谁是最敏捷的，于是它们进行了如下一个比赛。

首先，他们面前会有一排共 n 个数，它们比赛看谁能最先把每连续 k 个数中最大和最小值写下来，当然，这些机器人运算速度都很快，它们比赛的是谁写得快。

但是 Wind 也想知道答案，你能帮助他吗？

➤ **数据范围：** $k \leq n \leq 100000$

➔ 频繁出现的数值

➤ 问题描述(hdu 1806)

给定一个非降序排列的整数数组 $a_1, a_2, \dots, a_n$, 你的任务是对于一系列询问 (i, j) , 回答 $a_i, a_{i+1}, \dots, a_j$ 中出现次数最多的值所出现的次数。

➤ 问题分析

所有相等元素会聚在一起

我们把相等的一段用它的长度来记录

并记下每一个位置所在段的左端点和右端点

题目转换为求区间内的最大值, 成为了一个RMQ问题

→ Sound 静音问题

➤ 题目描述

数字录音中，声音是用表示空气压力的数字序列描述的，序列中的每个值称为一个采样，每个采样之间间隔一定的时间。

很多声音处理任务都需要将录到的声音分成由静音隔开的几段非静音段。为了避免分成过多或者过少的非静音段，静音通常是这样定义的： m 个采样的序列，该序列中采样的最大值和最小值之差不超过一个特定的阈值 c 。

请你写一个程序，检测 n 个采样中的静音。

➤ 问题分析 这道题就是求区间 $\max$,区间 $\min$ ，然后和 c 比较就好

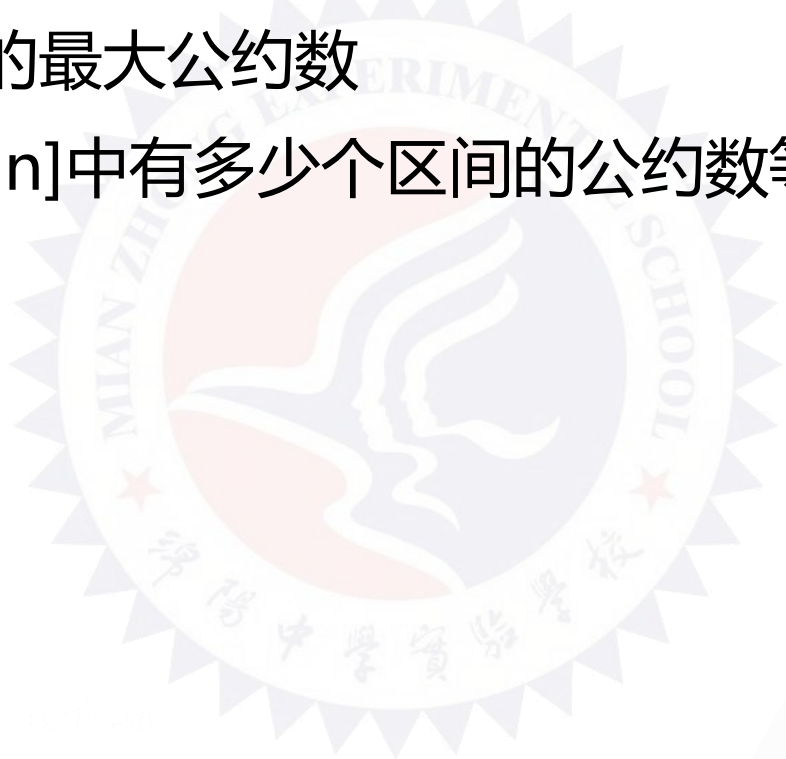


→ GCD

➤ 问题描述

一个长度为 n 的整数序列，给出 n 个询问区间
求指定区间的最大公约数

并求区间 $[1, n]$ 中有多少个区间的公约数等于求出来的这个公约数



➔ 与众不同

➤ 问题描述

A是某公司的CEO,每个月都会有员工把公司的盈利数据送给A,A是个与众不同的怪人,A不注重盈利还是亏本,而是喜欢研究“完美序列”:一段连续的序列满足序列中的数互不相同。

A想知道区间 $[L,R]$ 之间最长的完美序列。

➤ 数据范围 $N, M \leq 200000$

➤ 此题,暂时不讲,以后再说,太难了



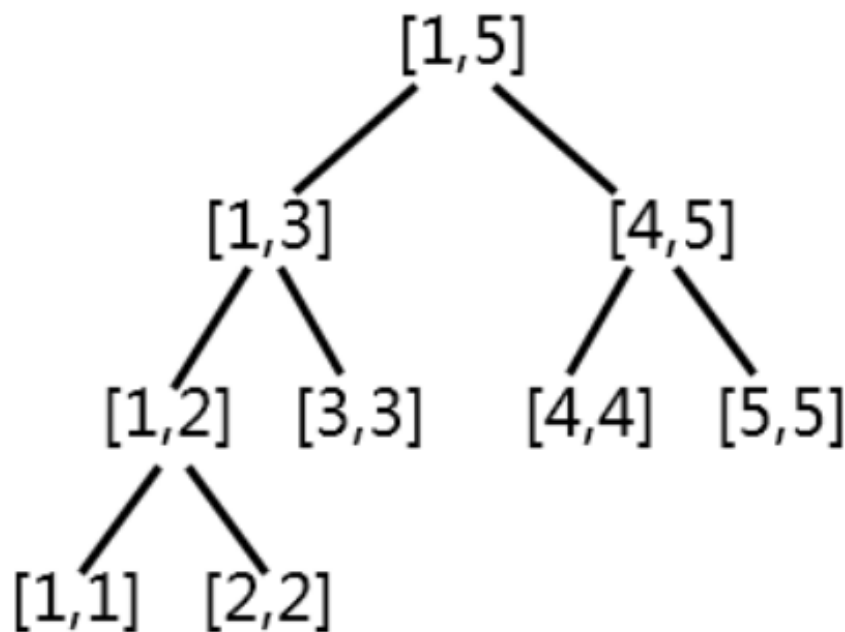
区间问题

二. 边查询边修改

- ① 点修改，求点的值
- ② 点修改，求区间 $[a,b]$ 之和
- ③ 点修改，求区间的极值
- ④ 区间修改，求点的值
- ⑤ 区间修改，求区间的极值或者和

→ 线段树--Interval Tree

线段树我习惯用英文称呼它 (Interval Tree)。
Interval 代表区间，恰恰说明了线段树是一种基于区间的数据结构。又称为区间树，每个结点代表的是一个区间，如图表示一个1~5区间的线段树结构。





线段树的常见操作

- 1、建立一棵线段树
- 2、查询某个区间
- 3、修改某个区间或点

NOI

➔ 经典问题

- 一个长度为N的一维数组($a[1] \sim a[N]$)
- 我们每次对该数组有一些操作:
 - 1、修改数组中某个元素的值
【1,5,4,1,6】 --- ($a[2]=3$) ---> 【1,3,4,1,6】
 - 2、修改数组中某段区间元素同时加上一个值
【1,5,4,1,6】 --- ($a[1] \sim a[3]+2$) ---> 【3,7,6,1,6】
 - 3、询问数组中某段区间的最大值
【1,5,4,1,6】 --- ($\max(1,4)=?$) ---> 5
 - 3、询问数组中某段区间的和
【1,5,4,1,6】 --- ($\text{sum}(3,5)=?$) ---> 11

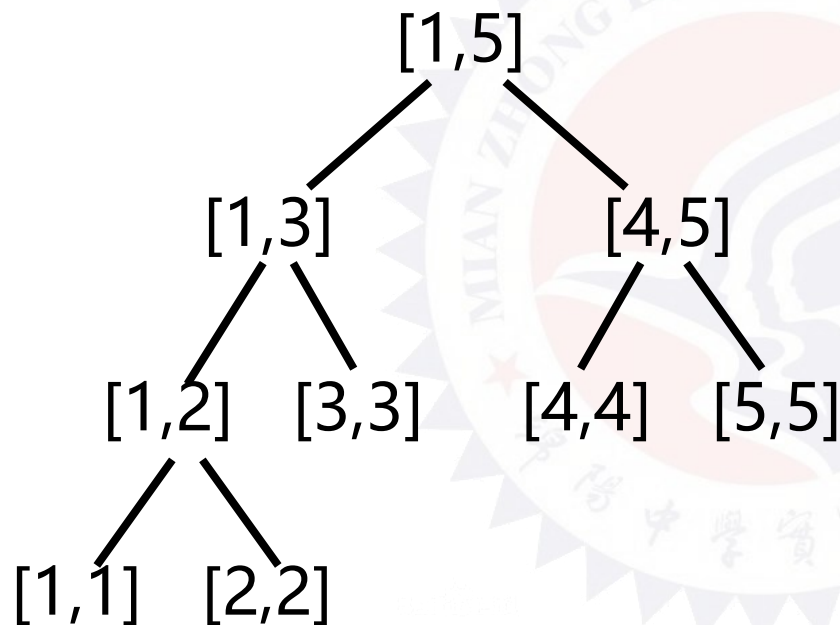


→ 线段树

- 如果只有一次询问？
 - 枚举相应区间内的元素，输出答案 $O(N)$
 - 更多的询问？
 - Q 次询问， $O(NQ)$ **That's too SLOW!**
- 线段树——在 $O(\log_2 N)$ 的时间内完成每次操作
 $O(Q \log_2 N)$
 - **Less than 1 seconds when $N=Q=100000$**

→ 线段树--结构

- 线段树的本质是一棵二叉树，不同于其它二叉树，线段树的每一个节点记录的是**一段区间**的信息。
- e.g. 对于长度为5的数组 $a[1] \sim a[5]$



对于任一非叶子节点，
若该区间为 $[L, R]$ ，则
左儿子为 $[L, (L+R)/2]$
右儿子为 $[(L+R)/2+1, R]$

如何建树？
如何记录信息？

➔ 建立一棵线段树

显然，线段树是递归定义的

对于一个线段树中的区间，若 $L < R$ ，则必能被分作 $[L, \text{Mid}]$ ， $[\text{Mid} + 1, R]$ ， $\text{Mid} = (L + R) \gg 1$ 。

若 $L = R$ ，则为递归的边界，此时该区间为一个点。

由此思想，建树时需要利用递归（或者模拟堆栈）。

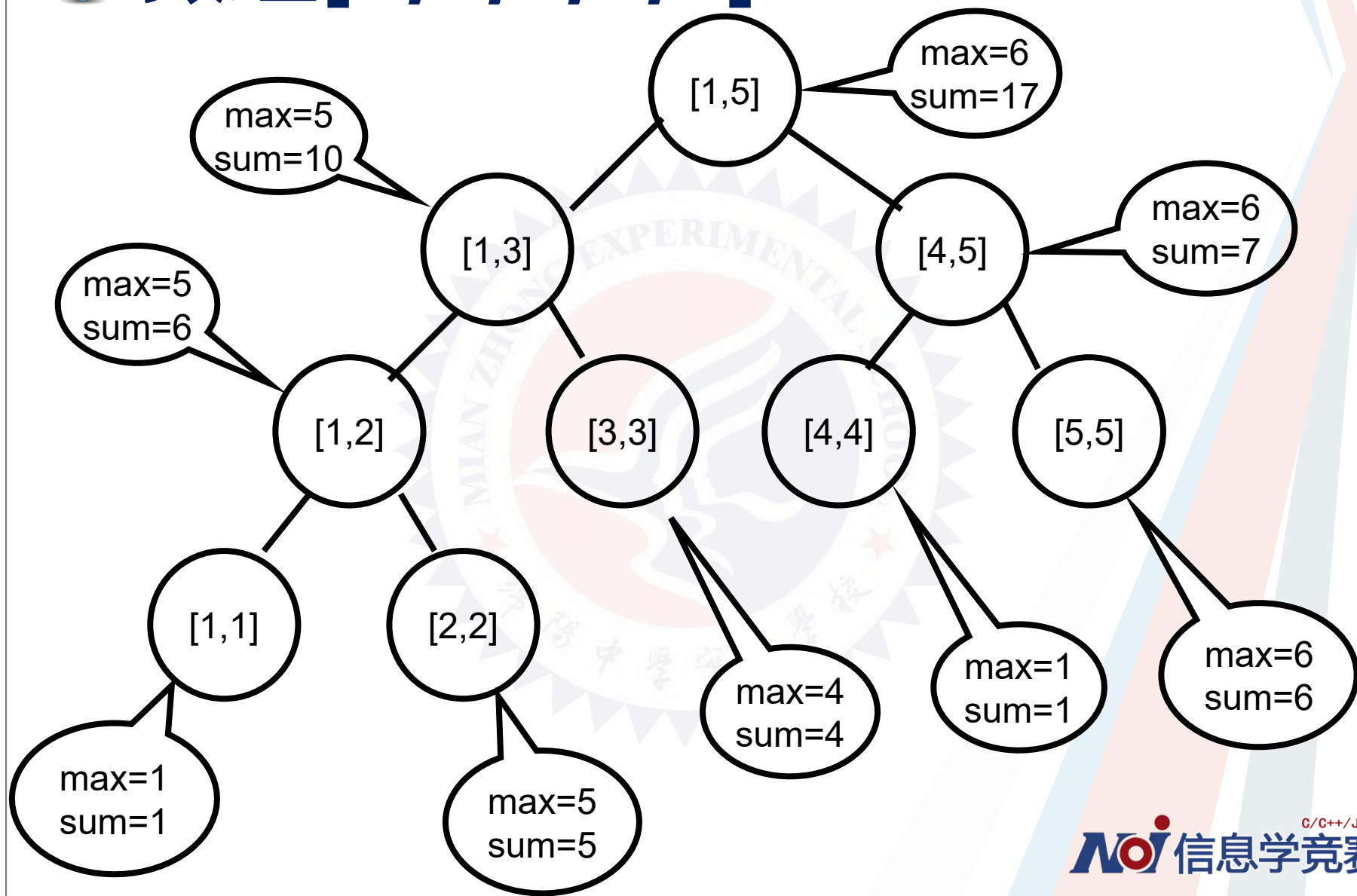
另外，由于这类线段树是二叉树式存储：

节点 T 的左儿子节点是 $T \gg 1$ ，代表 $[L, \text{Mid}]$ 区间。

节点 T 的右儿子节点是 $T \gg 1 + 1$ ，代表 $[\text{Mid} + 1, R]$ 区间。

这样便建立完了一棵线段树。

➔ 数组[1,5,4,1,6]



➔ 修改某个区间或点

线段树支持修改：

第一种是将某个点的值改变，并递归更新他的父亲。

第二种是将某个区间加上一个delta 值，或者改变为一个cover 值。

修改点：

显然，在 $[L,R]$ 中修改 $[LL,RR]$ ($LL==RR$) 的时候有三种情况。

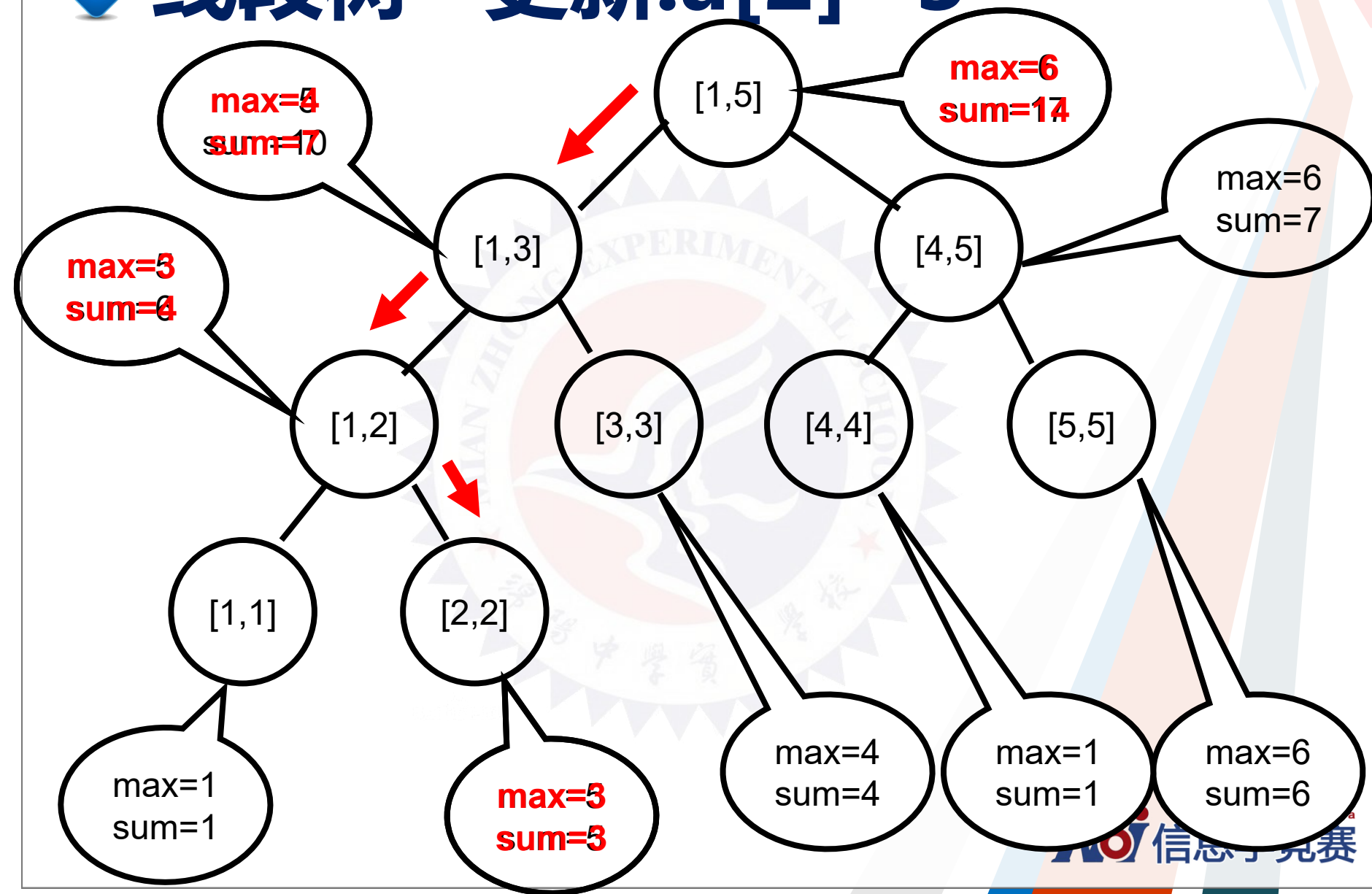
第一种， $[LL,RR]$ 被 $[L, Mid]$ 包含，此时直接在 $[L, Mid]$ 中修改 $[LL,RR]$ (在左子树)

第二种， $[LL,RR]$ 被 $[Mid+1, R]$ 包含，此时直接在 $[Mid+1, R]$ 中修改 $[LL,RR]$ (在右子树)

然后递归更新父亲。

不用考虑被截断（因为只是一个点）。

→ 线段树--更新:a[2]=3



→ 线段树---修改区间

显然，在 $[L,R]$ 中修改 $[LL,RR]$ ($LL < RR$) 的时候有三种情况。

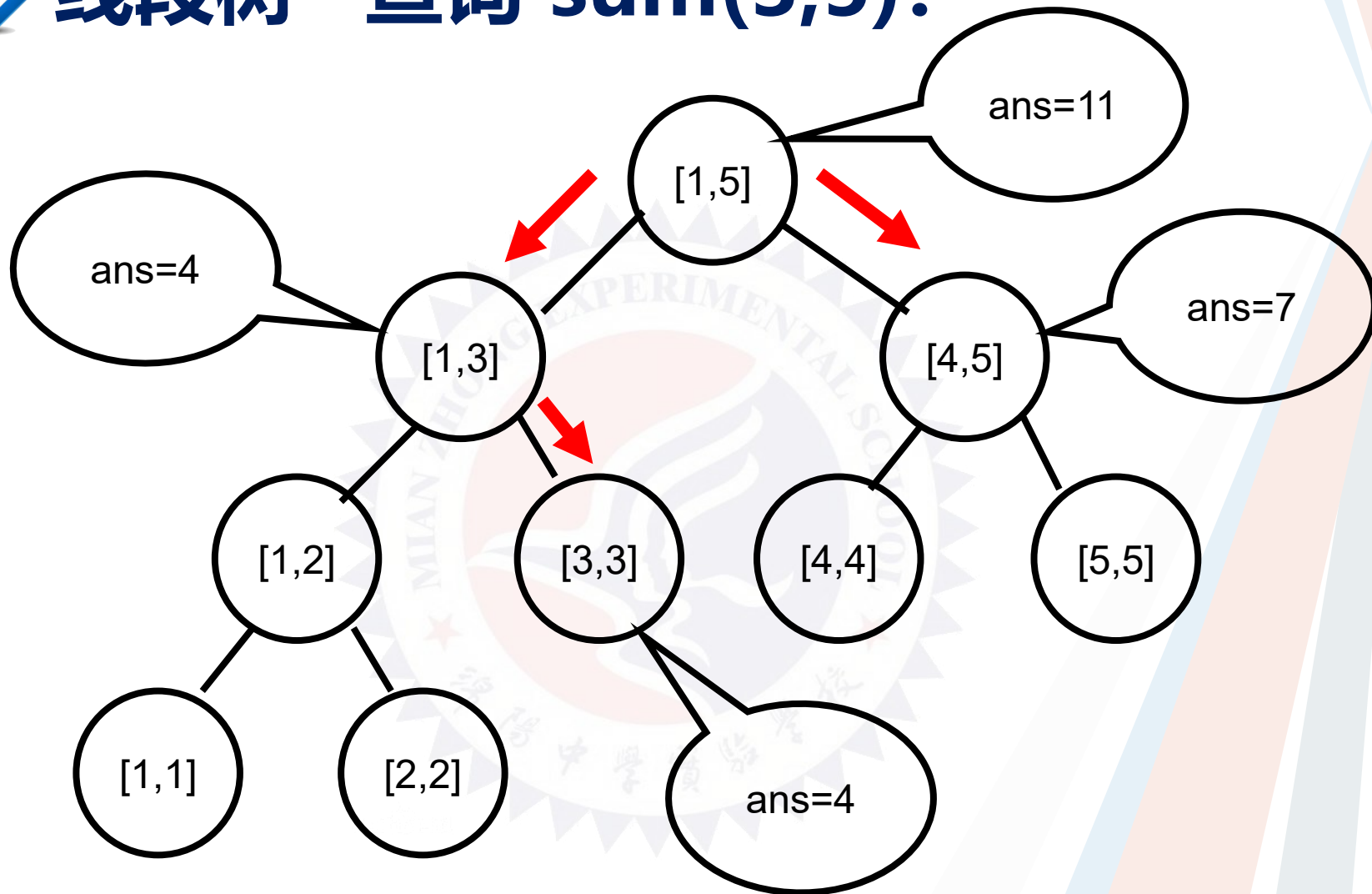
第一种， $[LL,RR]$ 被 $[L, \text{Mid}]$ 包含，此时直接在 $[L, \text{Mid}]$ 中修改 $[LL,RR]$ （在左子树）

第二种， $[LL,RR]$ 被 $[\text{Mid}+1, R]$ 包含，此时直接在 $[\text{Mid}+1, R]$ 中修改 $[LL,RR]$ （在右子树）

第三种，较复杂的一种， $[LL,RR]$ 被 Mid 从中间截断，此时在 $[L, \text{Mid}]$ 中修改 $[LL, \text{Mid}]$ ，在 $[\text{Mid}+1, R]$ 中修改 $[\text{Mid}+1, RR]$ ，然后将两个查询得到的结果进行Update，递归更新父亲。

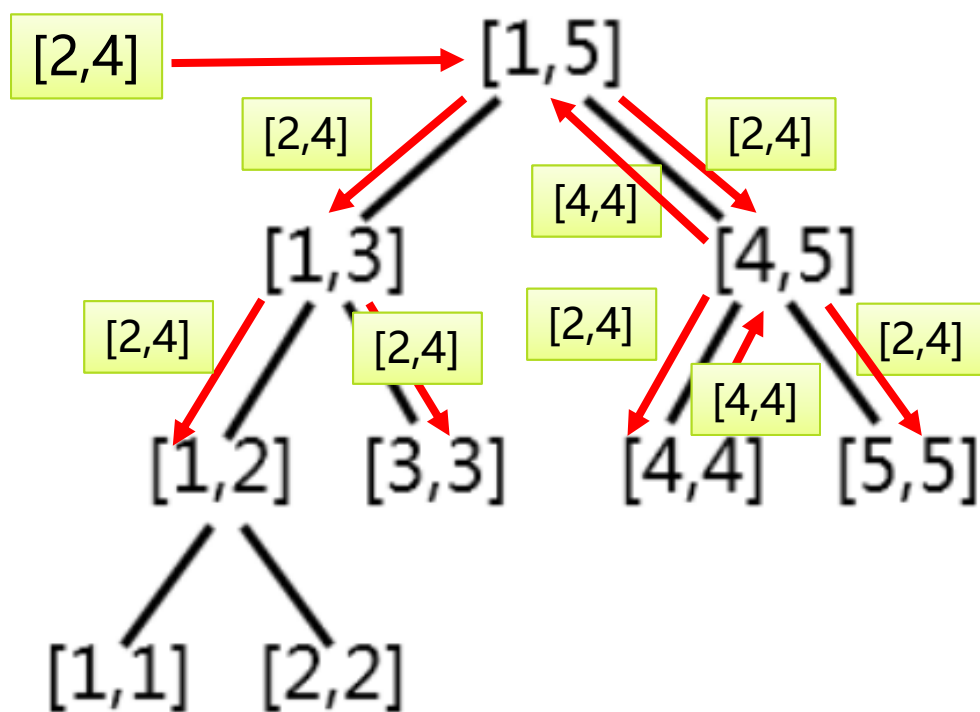
同时整段修改时，添加相应标记（原理见分块）

→ 线段树--查询 $\text{sum}(3,5)$?



线段树实现

线段树在实际代码实现时，为了避免每次把要处理线段分成几部分分情况讨论，我们采取把要处理线段整个区间下传，用线段树的点区间来更新答案，这样子代码实现更为简单和方便。例如我们在 $[1,5]$ 的区间处理一个 $[2,4]$ 的区间



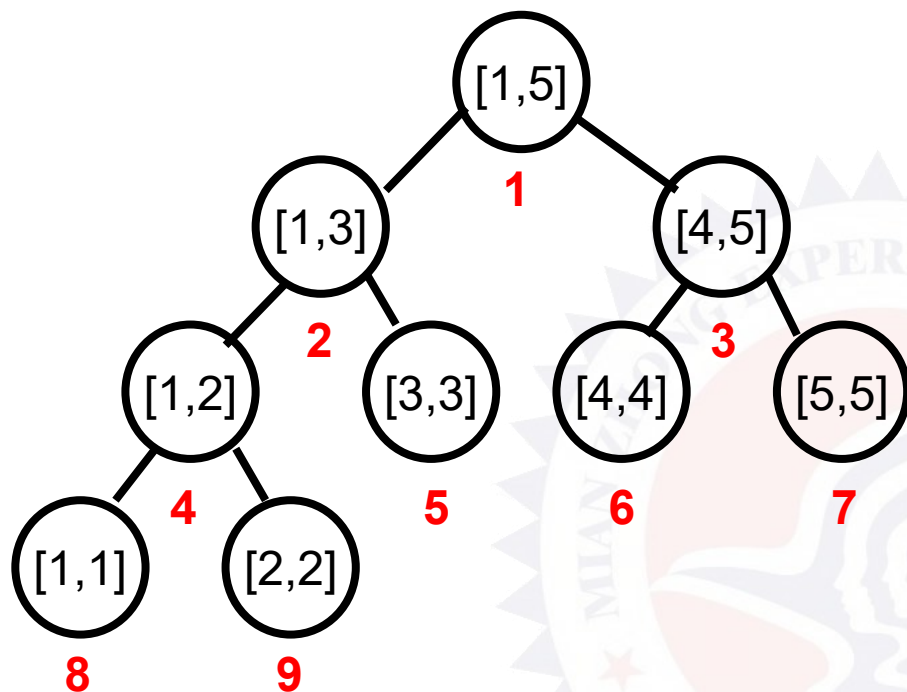
→ 线段树--实现

- 每一个节点记录的信息

```
struct node{  
    int left,right; //区间的端点  
    int max,sum; //视题目要求而定  
    int lazy;//标记  
};
```

- 整棵树: `node tree[maxn*4];`
- 如何记录左儿子和右儿子?

→ 线段树--实现



我们用一个数组 a 记录节点，且根节点的下标为1，

对于任一节点 $a[k]$ ，
它的左儿子为 $a[2*k]$
它的右儿子为 $a[2*k+1]$

一维结构体数组即实现了线段树节点的保存

→ 线段树--实现(建树)

➤ 建立一棵线段树，并记录原数组信息

```
inline void build(int id,int L,int R){//建一棵区间为L~R的线段树
    tree[id].lef=L; tree[id].rig=R; tree[id].sum=tree[id].lazy=0;
    if(L==R){ tree[id].sum=a[L]; return;}//叶子节点(一个点)
    int mid=(L+R)/2;
    build(id*2,L,mid);build(id*2+1,mid+1,R)//递归建左右子树
    //回溯时利用左右子树修改区间L~R的sum值
    tree[id].sum=tree[id*2].sum+tree[id*2+1].sum;
}
```

➤ 如果原数组从a[1]~a[n],调用build(1,1,n)即可

→ 线段树--实现(修改)

```

inline void down(int id){//标记下传函数
    tree[id*2].lazy+=tree[id].lazy;//标记下传到左子树
    tree[id*2].sum+=(tree[id*2].rig-tree[id*2].lef+1)*tree[id].lazy;
    tree[id*2+1].lazy+=tree[id].lazy;//标记下传到右子树
    tree[id*2+1].sum+=(tree[id*2+1].rig-tree[id*2+1].lef+1)*tree[id].lazy;
    tree[id].lazy=0;//清空标记
}
inline update(int id,int L,int R,int val){
    if(tree[id].lef>R || tree[id].rig<L) return;//处理区间与线段树区间无任何重合
    if(L<=tree[id].lef && tree[id].rig<=R)//线段树id点的区间整体被覆盖
    {
        tree[id].lazy+=val;
        tree[id].sum+=(tree[id].rig-tree[id].lef+1)*val;
        return;
    }
    if(tree[id].lazy>0) down(id);//递归左右子树之前，下传标记，反正要递归下去，节省时间
    update(id*2,L,R,val);update(id*2+1,L,R,val);//递归左右子树更新
    //回溯时利用左右子树修改区间L~R的sum值
    tree[id].sum=tree[id*2].sum+tree[id*2+1].sum;
}

```

→ 线段树--实现(查询)

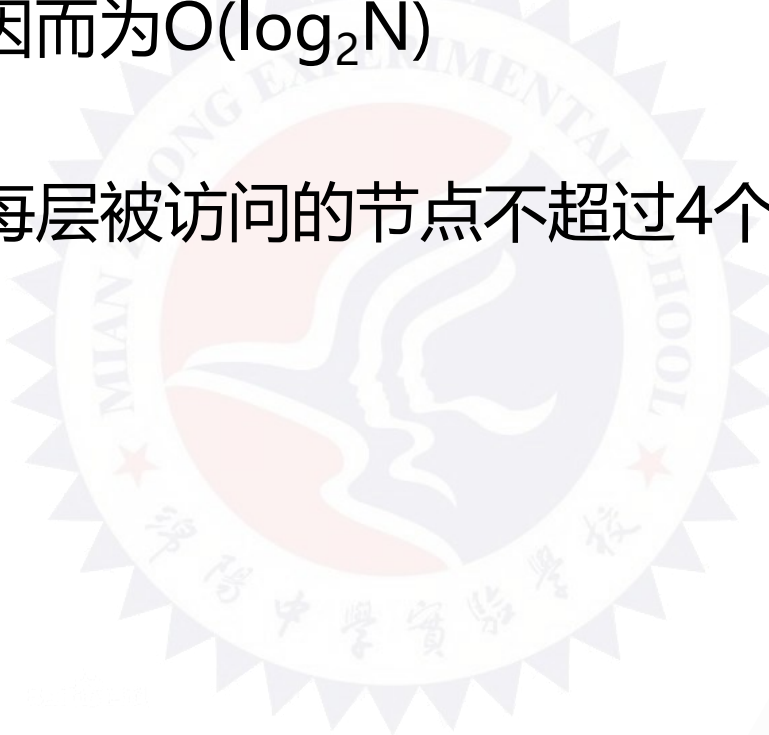
```
inline query(int id,int L,int R){// 查询函数
    if(tree[id].lef>R || tree[id].rig<L) return 0;
    if(L<=tree[id].lef && tree[id].rig<=R) return tree[id].sum;
    if(tree[id].lazy>0) down(id);
    return query(id*2,L,R)+query(id*2+1,L,R);
}
```





→ 线段树——时间复杂度

- 更新操作：由于总是一条路径从根到某个叶子，而树的深度为 $\log_2 N$,因而为 $O(\log_2 N)$
- 查询操作：每层被访问的节点不超过4个，因而同样为 $O(\log_2 N)$



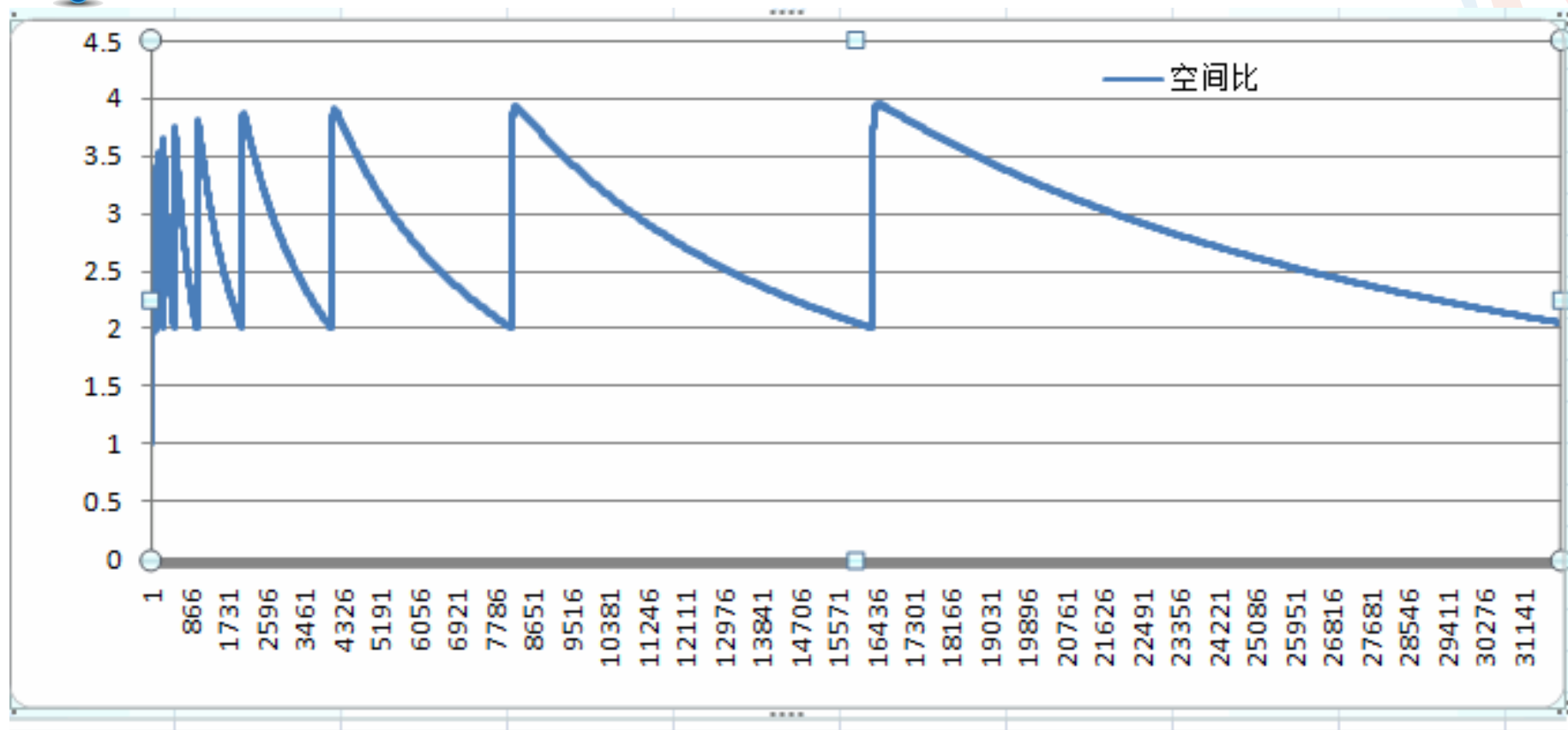


线段树——空间复杂度

- 设长度为 N 的数组在线段树中有 $F(N)$ 个节点
 - $F(N)$ 包括没有使用的下标为0的节点
 - 若 $N=2^n$, $F(N)=2^{(n+1)}$
 - 若 $N=2^{(n+1)}$, $F(N)=2^{(n+2)}$
- 因而对于 $2^n \leq N \leq 2^{(n+1)}$, 有
- $2^{(n+1)} \leq F(N) \leq 2^{(n+2)}$
- $F(N) \leq 4 * N$



线段树——空间复杂度 (II)



(图片转自<http://comzyh.tk/blog/archives/479/>)

线段树空间应开为原数组长度的4倍

→ 线段树应用

P3372 线段树 1

题目描述

如题，已知一个数列，你需要进行下面两种操作：

1.将某区间每一个数加上 x

2.求出某区间每一个数的和

操作1： 格式：1 x y k 含义：将区间 $[x,y]$ 内每个数加上 k

操作2： 格式：2 x y 含义：输出区间 $[x,y]$ 内每个数的和

数据规模：

对于30%的数据： $N \leq 8$ ， $M \leq 10$

对于70%的数据： $N \leq 1000$ ， $M \leq 10000$

对于100%的数据： $N \leq 100000$ ， $M \leq 100000$

(数据已经过加强 $\wedge$ ，保证在int64/long long数据范围内)



→ 线段树应用

P3373 线段树 2

题目描述

如题，已知一个数列，你需要进行下面三种操作：

- 1.将某区间每一个数乘上 x
- 2.将某区间每一个数加上 x
- 3.求出某区间每一个数的和

操作1： 格式： $1\ x\ y\ k$ 含义： 将区间 $[x,y]$ 内每个数乘上 k

操作2： 格式： $2\ x\ y\ k$ 含义： 将区间 $[x,y]$ 内每个数加上 k

操作3： 格式： $3\ x\ y$ 含义： 输出区间 $[x,y]$ 内每个数的和对 P 取模的结果

数据范围：

对于30%的数据： $N \leq 8$ ， $M \leq 10$

对于70%的数据： $N \leq 1000$ ， $M \leq 10000$

对于100%的数据： $N \leq 100000$ ， $M \leq 100000$



线段树应用

➤ 问题分析

本题难点在于如何处理+标记和*标记

我们来思考一下这个问题:

在 $a[i]$ 位置, 先 $+k$, 再 $*c$ 的结果: $ans = (a[i] + k) * c = a[i] * c + k * c$

在 $a[i]$ 位置, 先 $*c$, 再 $+k$ 的结果: $ans = a[i] * c + k$

通过分析会发现, 经过一次 $+k$ 标记, 则 $+k$ 标记的值累加 k 即可, 经过一次 $*c$ 标记, 则 $*c$ 运算 $*c$, 而加运算也要 $*c$ 。

所以对两个标记这样子维护即可。

注意下传标记时处理 $*c$ 标记要与上面做法一致

→ 集合

- 有三种类型的操作，
 1. "add x" 表示往集合里添加数x。
 2. "del x" 表示将集合中数x删除。
 3. "sum" 求出从小到大排列的集合中下标模5为3的数的和集合中的数都是唯一的。

分析：

把数作为下标，线段树中记录区间内模五为0...4的数字和，以及有多少个数

合并时，左子区间直接算，右子区间作相应位移



→ LIS

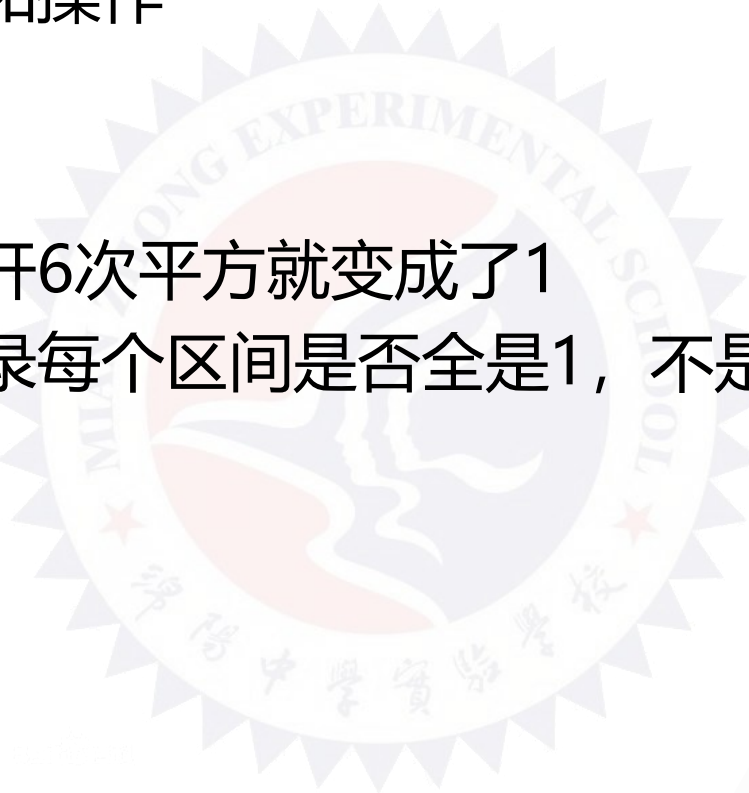
- 一个序列，可以区间加值，询问某一区间内连续最长上升子序列
- 分析：
- 对于每个区间，记录最长连续LIS
- 为了左右区间合并，要记录前缀最长及后缀最长LIS，还要记录端点值

NOI



上帝造题的七分钟

- 数列中的数 ($\leq 10^{12}$) 支持区间开平方 (下取整) 操作, 以及区间求和操作
- 分析:
- 序列中的数开6次平方就变成了1
- 所以可以记录每个区间是否全是1, 不是的话, 往下做





括号序列

- 给一个括号序列，可以区间改值，可以区间取反，询问区间是否为合法序列
- 分析：
- 将左右括号分别看作0,1,则序列的和为0，最小前缀和 ≥ 0
- 为此，维护序列和，最小前缀和，由于有取反，还要维护最大前缀和
- 两个标记setv[o], rev[o]先放下setv标志，再放下rev标志

→ 等差数列

- 序列的长度为250000，有四种操作
- 操作A，从st到ed这段区间内的数，分别加上 $1, 2, \dots, st - ed + 1$ 。
- 操作B，从st到ed这段区间内的数，分别加上， $st - ed + 1, st - ed, \dots, 1$ 。
- 操作C，将st到ed这段区间内的数赋值成x。
- 操作S，查询st到ed的这段区间内的数的总和。
- 分析：
- 操作C可用setv[o]操作符来进行
- 操作A，B都是加上一个等差数列，记录一段内的首项，末项即可



Sequence Operation

- 有N个为0或为1的数，有M个操作，操作有5种类型。
- (1) "0 a b" , 表示将区间[a,b]范围内的数全部置0。
- (2) "1 a b" , 表示将区间[a,b]内的数全部置1。
- (3) "2 a b", 表示将区间[a,b]内的数0变成1, 1变成0。
- (4) "3 a b", 表示查询[a,b]范围内1的数。
- (5) "4 a b", 表示查询[a,b]范围内最长的连续的1。
- 分析:
- 分治思想, 记录每一线段内最长的0,1序列, 最长的前缀、后缀0,1序列



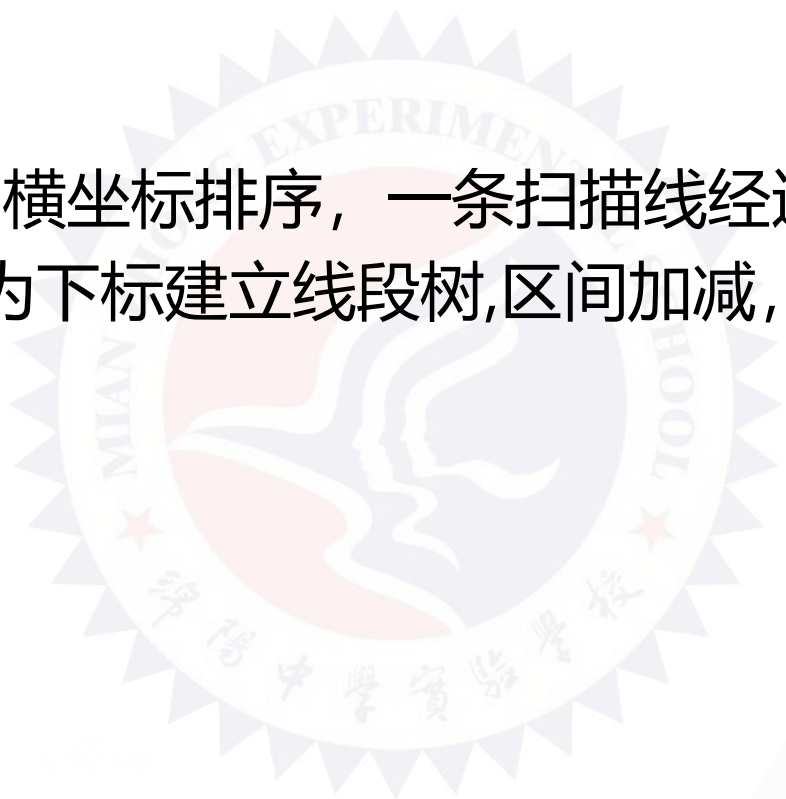
Little Elephant and Array

- 给定一个序列，若干个询问，区间 $[l,r]$ 内有多少值为 v 的数出现次数也为 v
- 分析：
- 离线处理所有询问，按终点从小到大排序
- $\text{cnt}[\text{value}]$ 表示 value 出现的次数， $\text{pos}[\text{value}]$ 表示 value 的各个位置
- 在线段树上单点标记上1,-1等值，区间统计即可



扫描线

- 给出N个矩形，求它们的面积并
- 分析：
- 按左下角的横坐标排序，一条扫描线经过
- 将y坐标作为下标建立线段树,区间加减，统计非0的长度





Necklace

- 有T组数据，每组数据有N个数，有Q个查询
- 每个查询要求区间 $[l,r]$ 之间数的和，但是出现多次的某个值只能算出现一次
- 分析：
- 离线处理，将所有询问按终点坐标从小到大排序
- $\text{cnt}[\text{value}]$ 表示value出现的次数， $\text{pos}[\text{value}]$ 表示value的各个位置
- 扫描时，最后一个位置的值加入线段树中，其他的位置都置为0



→ 借教室

- 每一天的教室数为 $a[i]$ ，第 j 个请求要在第 lj 天至第 rj 天借教室，
- 最多能满足前几个请求？
- 分析：
- 很容易想到的方法，线段树区间修改，区间统计最小值
- 但是线段树常数大，容易超时，且线段树编写较为麻烦
- 采用上题方法，每一天的请求打标记，起点+1，终点后一位-1
- 二分答案，打标记，从左往右扫描



→ 线段树应用

➤ luogu P4560 [IOI2014]Wall 砖墙

问题描述：

给定一个长度为 n 且初始值全为 0 的序列。你需要支持以下两种操作：

Add L, R, h : 将序列 $[L, R]$ 内所有值小于 h 的元素都赋为 h ，此时不改变高度大于 h 的元素值

Remove L, R, h : 将序列 $[L, R]$ 内所有值大于 h 的元素都赋为 h ，此时不改变高度小于 h 的元素值
你需要输出进行 k 次上述操作之后的序列。

The background features a central point from which several rays of varying colors (blue, orange, red, and grey) extend outwards. Faint binary code (0s and 1s) is scattered across the background, and a line graph with two data series (one blue, one orange) is visible on the right side.

Thank you!